

CS6886 — System Engineering for Deep Learning

Assignment 2: MobileNet-v2 on CIFAR-10

Hayagriv ED23B024

Question 1. Training Baseline

(a) Data Preparation and Augmentation

CIFAR-10 is loaded from the raw pickled batches via a custom **Dataset** (50,000 train / 10,000 test, $32 \times 32 \times 3$) and normalised per channel with the standard training-set statistics $\mu = (0.4914, 0.4822, 0.4465)$, $\sigma = (0.2470, 0.2435, 0.2616)$.

Split	Transform	Purpose
Train	<code>RandomCrop(32, padding=4, padding_mode='reflect')</code>	Translation invariance; reflect padding avoids the black borders zero padding would introduce.
Train	<code>RandomHorizontalFlip(p=0.5)</code>	Mirror invariance, label-preserving for all ten classes.
Train	<code>ToTensor(), Normalize(μ, σ)</code>	Zero-mean, unit-variance inputs.
Train	<code>RandomErasing(p=0.5, scale=(0.02,0.2), ratio=(0.3,3.3))</code>	Occlusion robustness; prevents reliance on one discriminative patch.
Test	<code>ToTensor(), Normalize(μ, σ)</code>	Deterministic; no augmentation.

`RandomErasing` is placed after `Normalize`, so the erased region is filled with zeros in normalised space the per-channel dataset mean in pixel space, not black. Erasing first would inject an out-of-distribution constant into every augmented image.

(b) Model Configuration and Training Strategy

Architecture. We use `mobilenet_v2` from `torchvision.models`, randomly initialised with `weights=None` and `num_classes=10`. The model has **2,236,682** parameters, a width multiplier of 1.0, and classifier dropout 0.2. BatchNorm uses the library defaults ($\epsilon = 10^{-5}$, momentum 0.1, affine, running statistics tracked). No pre-trained weights are used.

CIFAR-10 adaptation. Stock MobileNet-v2 downsamples by $32\times$, which on 32×32 inputs gives a 1×1 final feature map and destroys all spatial structure before global pooling. We set two strides from 2 to 1: `features[0][0]` (the stem convolution) and `features[2].conv[1][0]` (the

depthwise convolution of the first expansion-ratio-6 block). Total downsampling becomes $8\times$, giving a 4×4 feature map. No layers or channel counts are changed.

Component	Setting
Optimiser	SGD, Nesterov momentum 0.9
Base learning rate	0.05
LR schedule	Linear warmup (5 epochs, start factor 0.1)
Weight decay	5×10^{-4}
Loss	Cross-entropy, label smoothing $\varepsilon = 0.1$
Epochs / batch size	50 / 128 train, 256 test
Regularisation	Weight decay, label smoothing, dropout 0.2, RandomErasing
Checkpoint	Best-test-accuracy state dict
Hardware	Single NVIDIA P100 (Kaggle)

Table 1: Training configuration.

Rationale. Warmup prevents the large base learning rate from destabilising BatchNorm running statistics in the depthwise layers, which carry few parameters per channel. Cosine annealing to zero gives the fine convergence visible in the last ten epochs. Label smoothing discourages over-confident logits, which also narrows the activation dynamic range mildly helpful for the quantisation that follows.

(c) Results, Curves, and Failure Modes

Accuracy. The model reaches **94.28% test top-1**, with training accuracy 97.34%. This is the best epoch checkpoint, which is the artefact saved during training and carried forward to next questions; the final-epoch value was 94.07%. I reported the checkpoint figure throughout so that the compression results are stated against exactly the model they were measured on.

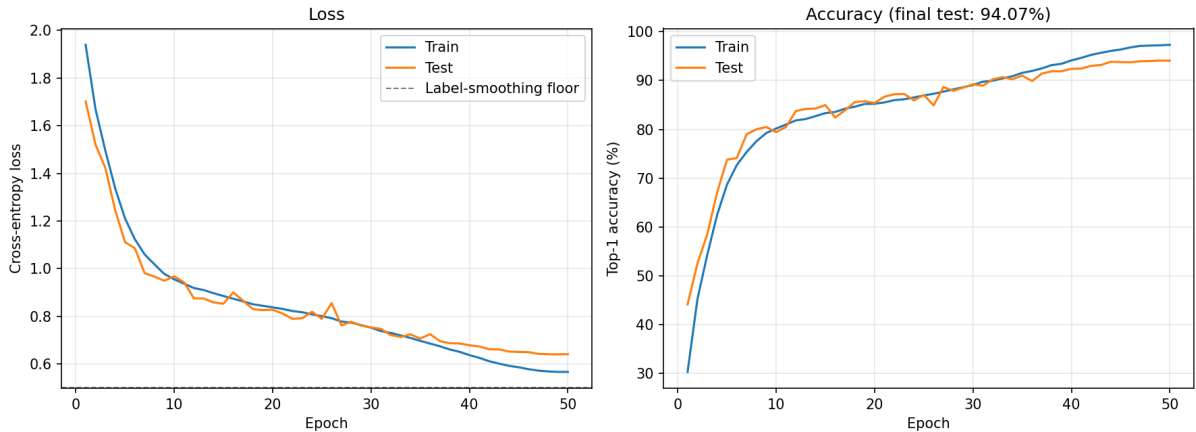


Figure 1: Training and test loss (left) and top-1 accuracy (right) over 50 epochs.

Generalisation gap. The train test gap is ≈ 3.1 points, a healthy margin. A much larger gap would indicate insufficient regularisation, a near-zero gap that the augmentation is suppressing usable capacity. Both curves flatten over the last five epochs, so the model has converged for this schedule and further gains would need a longer schedule rather than more epochs at this learning rate.

Failure modes. The residual error is dominated by class pairs that are not separable at this input resolution:

- **cat/dog** — the cues distinguishing them (ear geometry, snout length, whiskers) span only a few pixels at 32×32 , so this is a resolution limit rather than a capacity limit.
- **bird/airplane** and **deer/horse** — shared silhouette and shared background context (sky, grassland) rather than object identity.
- **Animal vs vehicle** — rigid, high-contrast shapes survive downsampling better than articulated, textured subjects with variable pose.
- **Augmentation noise** — RandomErasing occasionally removes the only discriminative region of a small centred subject, producing training samples with no usable signal. An accepted cost of the regularisation.

Architectural limits relevant to compression. Two properties of MobileNet-v2 shape the design here. The inverted residual blocks expand channels $6\times$ and apply ReLU6, producing activation distributions with long tails and widely differing dynamic ranges across channels within the same tensor. The linear bottleneck projections have no non-linearity at all, so their outputs are unbounded and not naturally clipped. Both make a single per-tensor scale a poor fit, which is why we adopt per-channel weight quantisation with calibrated activation ranges.

Question 2. Model Compression Implementation

The pipeline is uniform quantization with BatchNorm folding, implemented from scratch (no `torch.quantization` or any other compression library call). All bit-widths and scheme choices are command-line flags.

(a) Compression Method and Design Choices

Weights: symmetric, per-output-channel. For a weight tensor W with output channel c and bit-width b_w :

$$s_c = \frac{\max_i |W_{c,i}|}{2^{b_w-1} - 1}, \quad q_{c,i} = \text{clamp}\left(\left\lceil \frac{W_{c,i}}{s_c} \right\rceil, -2^{b_w-1}, 2^{b_w-1} - 1\right), \quad \hat{W}_{c,i} = s_c q_{c,i}.$$

Symmetric, no zero point: trained weights are approximately zero-centred, so a zero point buys little range, and omitting it reduces the integer convolution to a plain dot product with one output rescale.

Per-channel, not per-tensor: MobileNet-v2’s depthwise convolutions apply one filter per channel independently, and their per-channel weight magnitudes span one to two orders of magnitude within a single tensor. Under a shared scale the small-magnitude channels collapse to one or two levels. Per-channel scales remove this for a few bytes per channel.

Activations: asymmetric, per-tensor, calibrated. For observed range $[\alpha, \beta]$ and bit-width b_a :

$$s = \frac{\beta - \alpha}{2^{b_a} - 1}, \quad z = \left\lceil -\frac{\alpha}{s} \right\rceil, \quad \hat{x} = s \left(\text{clamp}\left(\left\lceil \frac{x}{s} \right\rceil + z, 0, 2^{b_a} - 1\right) - z \right).$$

Asymmetric because ReLU6 outputs are non-negative, so a symmetric scheme would waste half its range. The range is widened to include zero so padding and post-ReLU zeros stay exactly representable. Quantization is applied at layer *inputs*, so each intermediate tensor is quantized exactly once and the activation accounting in Question 4 does not double count.

Range calibration. Ranges are estimated from 8 batches (1024 images) of *training* data with observers attached; the test set is never used. We take the 99.9th percentile with an EMA across batches rather than absolute min/max, because the $6\times$ expansion layers emit rare large activations that would otherwise stretch the scale and collapse the bulk of the distribution into a few levels. The percentile is configurable.

BatchNorm folding. Every BatchNorm is folded into its preceding convolution:

$$W' = W \cdot \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, \quad b' = (b - \mu) \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

Exact at inference. It removes all BatchNorm parameters and running statistics from the stored model and matches deployment, where BatchNorm is never a separate runtime operator. It also multiplies each output channel by a different γ/σ , *widening* the per-channel magnitude spread so folding and per-channel quantization are coupled choices, not independent ones.

Configurability.

Flag	Effect
-weight_quant_bits	b_w , weight bit-width
-activation_quant_bits	b_a , activation bit-width
-per_channel / -per_tensor	weight scale granularity
-no_fold_bn	disable BatchNorm folding
-no_act_quant	weight-only compression
-protect_sensitive	keep stem and classifier at 8 bits
-percentile	activation calibration percentile

(b) Application to MobileNet-v2

Every Conv2d and Linear layer is quantized 52 convolutions and 1 fully-connected layer, covering 100% of the weight parameters.

Location	Layer type	Count	Bits
features[0][0]	3×3 stem convolution	1	8
features[1]	depthwise + project	2	b_w
features[2..17]	expand + depthwise + project per block	48	b_w
features[18][0]	final 1×1 convolution	1	b_w
classifier[1]	fully-connected, 10 outputs	1	8
Total		53	

Exceptions.

- **BatchNorm is not quantized it is eliminated** by folding, so no BatchNorm value is stored at any precision. Quantizing it directly would be harmful: γ and σ sit on a division, so error is amplified rather than averaged away.
- **Stem and classifier stay at 8 bits** even when $b_w < 8$. The stem sees the raw image and the classifier emits logits directly, both without downstream processing to absorb error. Together they hold under 2% of parameters.
- **Folded biases stay FP32.** They sit on the accumulator path with no subsequent normalisation, and cost $\approx 0.8\%$ of the parameter count.

- **Residual adds and ReLU6 stay in float.** Their inputs are already quantized values, so requantizing adds error without saving storage or traffic.

Numerical simulation. Accuracy is measured by fake quantization quantize then dequantize in float which reproduces exactly the error an integer kernel would produce while running on ordinary float kernels. Stored size is computed analytically from bit-widths and metadata. Accuracy is thus measured empirically and size computed exactly, with neither confounded by the efficiency of any particular integer kernel.

(c) Storage Overheads

A naive $n \times b_w / 8$ estimate ignores everything a quantized model must carry alongside its payload.

Component	Cost	Justification
Weight payload	$n \cdot b_w$ bits	The quantized indices.
Weight scales	$16 C_{\text{out}}$ bits/layer	One FP16 scale per output channel; FP16 is ample since a scale only sets a dynamic range.
Folded biases	$32 C_{\text{out}}$ bits/layer	Kept in FP32, per (b).
Activation qparams	64 bits/tensor	One FP32 scale and one INT32 zero point, fixed after calibration.

The FP32 baseline counts *both* parameters and BatchNorm running statistics, since both must ship to reproduce the float model:

$$S_{\text{FP32}} = 32 \left(\sum_i n_i + n_{\text{BN buffers}} \right), \quad S_{\text{quant}} = \sum_{\ell} \left(n_{\ell} b_w + 48 C_{\text{out}}^{\ell} + 64 \right) \text{ bits}.$$

BatchNorm folding is itself a compression step: it removes 68,224 float values, 266.5 KB at FP32, more than every metadata category combined.

Question 3. Compression Results

(a) Sweep Configuration

I sweep weight bit-width $b_w \in \{8, 6, 5, 4, 3, 2\}$ against activation bit-width $b_a \in \{8, 6, 4\}$, holding the scheme fixed (BN folded, per-channel weight scales, 99.9th-percentile activation calibration, stem and classifier pinned at 8 bits). Four further runs ablate one design choice each. All 22 runs use seed 42 and the same checkpoint, so differences are attributable to configuration alone.

Baseline. The compressed artefact is the checkpoint saved during training, which scores **94.28%** top-1 and occupies **8.662 MB** in FP32 (2,236,682 parameters plus 34,112 BatchNorm running statistics). All accuracy deltas below are relative to this figure.

(b) Results

Weights and activations fail differently. Weight width degrades smoothly: $8 \rightarrow 6 \rightarrow 5$ bits costs under 0.4 points in total, 4 bits costs 1.9, and the model collapses between 3 and 2 bits. Activation width instead shows a cliff. At 6 bits the penalty is ≈ 0.5 points regardless of

Config	b_w	b_a	Top-1 (%)	Δ (pts)	Ratio	Size (MB)
FP32 baseline	32	32	94.28	—	1.00	8.662
w8a8	8	8	94.13	−0.15	3.94	2.199
w8a6	8	6	93.78	−0.50	3.94	2.199
w8a4	8	4	85.84	−8.44	3.94	2.199
w6a8	6	8	94.16	−0.12	5.17	1.677
w6a6	6	6	93.78	−0.50	5.17	1.677
w6a4	6	4	85.26	−9.02	5.17	1.677
w5a8	5	8	93.94	−0.34	6.12	1.416
w5a6	5	6	93.75	−0.53	6.12	1.416
w5a4	5	4	85.92	−8.36	6.12	1.416
w4a8	4	8	92.39	−1.89	7.50	1.155
w4a6	4	6	91.72	−2.56	7.50	1.155
w4a4	4	4	80.00	−14.28	7.50	1.155
w3a8	3	8	78.96	−15.32	9.69	0.894
w3a6	3	6	77.31	−16.97	9.69	0.894
w3a4	3	4	61.17	−33.11	9.69	0.894
w2a8	2	8	10.47	−83.81	13.69	0.633
w2a6	2	6	10.41	−83.87	13.69	0.633
w2a4	2	4	10.11	−84.17	13.69	0.633
<i>Ablations</i>						
w4a8, per-tensor scales	4	8	78.03	−16.25	7.72	1.122
w4a8, no layer protection	4	8	92.23	−2.05	7.54	1.148
w4a32, weights only	4	32	92.49	−1.79	7.50	1.154
w8a8, per-tensor, no protect	8	8	94.05	−0.23	4.00	2.166

Table 2: Compression sweep over 22 configurations. Ratio and size are for the full quantized model including all metadata. The adopted operating point is in bold.

b_w ; at 4 bits it is 8–9 points, again regardless of b_w . The $a4$ column is essentially flat across weight widths (85.84, 85.26, 85.92 at $b_w = 8, 6, 5$), showing the loss there comes from activation clipping alone and does not interact with the weights.

Model size is independent of activation bit-width. All three rows at a given b_w report the same ratio and size, because activations are never stored they exist only during a forward pass.

Per-channel scaling matters more as bit-width falls. The benefit of per-channel weight scales is strongly bit-width dependent. At 8 bits, per-tensor quantization is nearly free (94.05% vs 94.13%, a gap of 0.08 points). At 4 bits the same change costs 14.4 points (78.03% vs 92.39%). At 8 bits a shared scale still leaves every depthwise channel several distinct levels; at 4 bits the small-magnitude channels collapse to one or two. Per-channel scaling is thus not a general-purpose improvement but the specific enabler of sub-8-bit quantization on MobileNet-v2, and it costs only 33 KB.

Activation quantization is nearly free at 8 bits. Comparing $w4/a32$ (92.49%) with $w4/a8$ (92.39%), quantizing activations to 8 bits costs 0.10 points while cutting activation memory traffic fourfold.

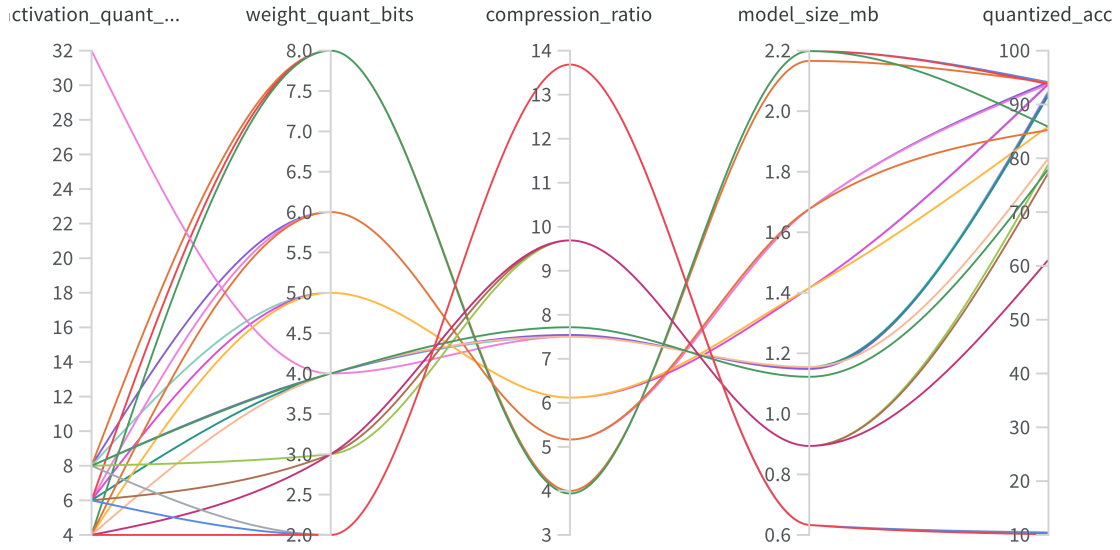


Figure 2: Wandb parallel coordinates plot over the 22 sweep runs.

Protecting the sensitive layers is close to free. Holding the stem convolution and classifier at 8 bits recovers 0.16 points (92.39% vs 92.23%) while reducing the ratio only from $7.54\times$ to $7.50\times$ under 0.6% of the compressed size.

Selected operating point. We adopt **w5/a6**. It sits at the knee of the trade-off: relative to the naive per-tensor INT8 configuration it delivers 53% more compression ($6.12\times$ vs $4.00\times$) for 0.30 points of accuracy, and relative to *w4/a6* it recovers 2.03 points for 23% less compression. Below $b_w = 4$ or $b_a = 6$ accuracy falls faster than size, so post-training quantization without fine-tuning is not viable there.

Question 4. Compression Analysis

All figures below are for the single adopted configuration: **5-bit per-channel weights, 6-bit activations**, BatchNorm folded, stem convolution and classifier held at 8 bits, activation ranges calibrated at the 99.9th percentile on 1024 training images.

Metric	Value
Weight compression ratio	$6.57\times$
Activation compression ratio	$5.33\times$
Accuracy (quantized)	93.75%
Accuracy (FP32 baseline)	94.28%
Accuracy drop	0.53 points
Final model size	1.416 MB
Model compression ratio	$6.12\times$

Table 3: Headline results at *w5/a6*.

(a) Weight Compression Ratio

6.57×, defined as the full FP32 model (8870.3 KB, i.e. parameters plus BatchNorm running statistics) divided by the stored weight payload (1349.3 KB).

The effective cost is 5.02 bits per weight: 2,202,568 weights are stored, of which 13,664 (stem and classifier) are held at 8 bits and the remainder at 5.

(b) Activation Compression Ratio

5.33×.

How activations were measured. Activations are not stored with the model, so they are measured as the tensors that cross quantized layer boundaries during a single forward pass at batch size 1. Because our scheme quantizes each layer’s *input*, every intermediate tensor is counted exactly once, with no double counting across consecutive layers. Concretely, an observer on each of the 53 quantized layers records the element count of its input tensor. We have two figures:

Measure	Elements	FP32	6-bit
Total across 53 tensors (one forward pass)	1,623,808	6343.0 KB	1189.7 KB
Peak single tensor (min. buffer)	147,456	576.0 KB	108.0 KB

The *total* figure (5.33×) is the activation memory traffic of one inference and is the number reported. The *peak* figure (5.33×) is the smallest buffer a layer-by-layer executor must hold. The two ratios coincide at $32/6 = 5.33$ because activation metadata is negligible: one FP32 scale and one INT32 zero point per tensor, 424 bytes across the whole network.

(c) Accuracy of the Quantized Model

93.75% top-1 on the CIFAR-10 test set, against 94.28% for the FP32 checkpoint. A drop of 0.53 points, or 0.56% relative. No fine-tuning or quantization-aware training was used; this is post-training quantization applied directly to the trained checkpoint, with calibration performed on training data only.

(d) Final Model Size

1.416 MB, down from 8.662 MB, a model compression ratio of **6.12×**.

Question 5. Reproducibility & Repository

(a) Codebase Structure

Training, evaluation, and compression are separate modules. `evaluate.py` is imported by both compression entry points, so FP32 and quantized accuracy are measured through an identical code path.

File	Role	Contents
data.py	shared	Dataset, transforms, loaders
train.py	training	Q1 training loop, schedule, curves
evaluate.py	evaluation	Top-1, confusion matrix, per-class
test.py	compression	Single configuration + size report
sweep.py	compression	Q3 sweep, ablations, wandb logging
compression/quantizers.py		Quantization primitives
compression/bn_fold.py		BatchNorm folding
compression/quant_model.py		Wrappers, surgery, calibration
compression/size_accounting.py		Storage accounting

(b) Environment, Commands, and Seeds

Environment. Python 3.10, single NVIDIA P100 (Kaggle), CUDA 12.1. Pinned in requirements.txt: torch==2.4.0, torchvision==0.19.0, numpy==1.26.4, Pillow==10.4.0, matplotlib==3.7.5, wandb==0.17.7.

Seeds. Every entry point calls `set_seed(42)`, seeding `random`, `numpy`, `torch` and `torch.cuda`, and setting `torch.backends.cudnn.deterministic = True`, `torch.backends.cudnn.benchmark = False`. Overridable with `-seed`.

Commands.

Step	Command
Install	<code>pip install -r requirements.txt</code>
Train (Q1)	<code>python train.py --epochs 50 --seed 42</code>
Compress (Q2)	<code>python test.py --weight_quant_bits 8</code> <code> --activation_quant_bits 8</code>
Sweep (Q3)	<code>python sweep.py --wandb</code>
Q4 point	<code>python test.py --weight_quant_bits 5</code> <code> --activation_quant_bits 6</code>

(c) Repository Link

https://github.com/Hayagriv1/CS6886_Assignment_2